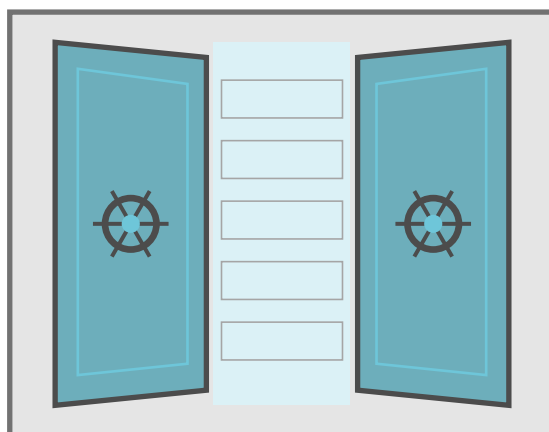


Criptografía y Seguridad

Probando la seguridad Penetration testing



Apunte integral — Teoría

Clase 10b · Capítulo 23 (secciones 1–2), *Computer Security: Art and Science* (M. Bishop)

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de la transcripción de la clase teórica (Prof. Leandro Suárez) y de las diapositivas oficiales de la cátedra. Esta clase no tiene clase práctica asociada.

1 Introducción y contexto

Esta clase responde a una pregunta de fondo: **¿cómo verificamos que un sistema es seguro?** Hay varias formas de corroborar el funcionamiento correcto de un sistema; la clase contrapone dos de ellas —la **verificación formal** y la **prueba de penetración** (*penetration testing* o *pentesting*)— y luego profundiza en esta última: su metodología, sus etapas y dos ejemplos concretos.

Encuadre del profesor

El *pentesting* había quedado pendiente de la clase anterior, donde sólo se habían “vislumbrado” los conceptos de *red team*, *breach & attack simulation* y las etapas de un pentest. El profesor aclara que **todo esto entra en el parcial**: “Igual, a ver, esto de la segunda parte entra todo en el parcial; así que es lo mismo que lo veamos hoy o la semana pasada.”

— Prof. Leandro Suárez, en clase

La gran conclusión que atraviesa toda la clase

El *testing* puede mostrar la **presencia** de bugs/vulnerabilidades, pero **nunca prueba su ausencia** (idea atribuida en clase a Dijkstra). El pentesting *parte de la premisa de que ya existen vulnerabilidades* e intenta demostrarlas; que no encuentre nada **no** significa que el sistema sea seguro, sólo que no se comprometió *bajo las condiciones de esa prueba*.

2 Verificación formal

Verificación formal

Verificación **matemática** de que un sistema cumple con ciertas restricciones (las que nosotros le imponemos). Se modela el sistema mediante:

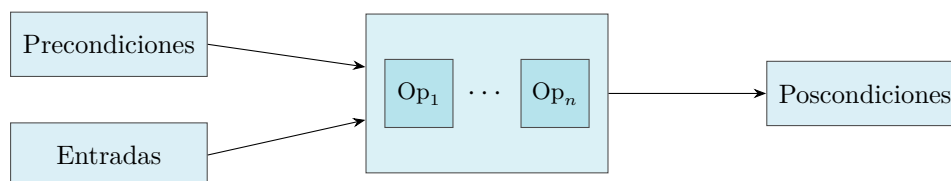
- **Precondiciones**: una hipótesis sobre el *estado* del sistema (p. ej. “el sistema está en un estado seguro”).
- **Poscondiciones**: el resultado de aplicar las operaciones del sistema a un *input dado*.

Requerimiento: las poscondiciones deben cumplir las restricciones impuestas.

Es el clásico esquema de prueba matemática: se parte de una hipótesis (precondición), se ejecuta la verificación y se corrobora que las poscondiciones cumplen lo que se pedía. Está orientada a *probar matemáticamente que un sistema hace lo que dice hacer*.

Diagrama del modelo

La siguiente figura recrea el esquema de la diapositiva: las *precondiciones* y las *entradas* alimentan una secuencia de operaciones $Op_1, \dots, Op_n$ del sistema, que produce las *poscondiciones*.



¿Por qué casi no se usa en la práctica?

La verificación formal es **muy potente pero impráctica** para sistemas reales:

- Es **extremadamente compleja**: con sólo un par de variables y operaciones ya cuesta seguirla; hay que mapear todas las posibilidades, precondiciones e interacciones entre componentes.
- Es **muy costosa** en tiempo: a veces lleva más tiempo verificar el sistema que construirlo de nuevo.
- Los sistemas **evolucionan muy rápido**: antes de terminar la verificación, el sistema ya cambió y hay que rehacerla.

El profesor la compara con las viejas “pruebas de escritorio”: probar una función con distintos inputs (incluido el caso **null**) es realizable; probar *un sistema completo* de esta forma se vuelve impracticable.

3 Prueba de penetración (*Penetration testing*)

Prueba de penetración

Prueba (no demostración matemática) para verificar que un sistema cumple con ciertas restricciones. Se modela con el mismo lenguaje de pre/poscondiciones que la verificación formal, pero con un giro clave:

- **Precondiciones**: hipótesis sobre el estado del sistema **y la existencia de una vulnerabilidad**. *Se asume de entrada que la vulnerabilidad existe.*
- **Poscondiciones**: **sistema comprometido**.
- **Ejecución**: aplicar pruebas para intentar *mover al sistema del estado inicial al estado comprometido*.

Si la ejecución logra ese movimiento, se concluye que el sistema *está comprometido* bajo esas condiciones. Es una forma **mucho más económica, simple y eficiente** de probar un sistema que la verificación formal, y es la que usan las empresas (grandes o PyMEs). Permite, además, enfocar primero los componentes más críticos de un sistema que evoluciona rápido.

No encontrar nada $\neq$ ser seguro

Que la poscondición “no se cumpla” (no encontré la vulnerabilidad) **no significa** que no haya fallas: puede haber otro vector de ataque o vectores que esa prueba no exploró. La prueba de penetración *no prueba la ausencia* de vulnerabilidades.

4 Similitudes y diferencias

	Prueba de penetración	Verificación formal
Prueba la existencia de vulnerabilidades	Sí	Sí
Prueba la ausencia de vulnerabilidades	NO	Sí, pero debe incluir TODOS los factores externos
Costo / practicidad	Económica, eficiente, práctica	Muy costosa, a menudo impráctica

Ambas técnicas **prueban la existencia** de vulnerabilidades. La diferencia está en la *ausencia*: la verificación formal sí puede probarla, pero a condición de incluir *todos* los factores posibles, incluidos los externos. La prueba de penetración no puede.

Lo que se ignora en la práctica

En la práctica, incluso una verificación formal prueba la ausencia de vulnerabilidades en **un algoritmo, programa o ambiente determinado**, y **no en un sistema completo**. Se ignoran cuestiones como la **instalación** y el **uso**, el sistema operativo sobre el que corre, el estado y el lugar donde está instalado, etc. Esto conecta con la frase de Dijkstra repetida en clase: el testing muestra la presencia de bugs, no su ausencia.

5 Objetivos del pentesting

El objetivo central es **probar la eficacia de los controles de seguridad de un sistema**. Esto se logra:

- **Intentando violar la política de seguridad** ya existente. Por eso suele haber un *equipo designado* que conoce de seguridad y entiende las implicancias.
- **Ejecutando técnicas similares a las de un atacante**: simular ser un atacante. Esto se conoce como **ethical hacking**.
- De forma **análoga a las pruebas manuales** de un sistema: alguien que no participó de la construcción intenta romperlo (pasar inputs inválidos, ver cómo reacciona). *No reemplaza un buen diseño e implementación*.
- Probando **al sistema como un todo**, no sólo sus aspectos técnicos.

Ética del hacking: comportamiento del pentester

Al simular ser un atacante, el pentester debe adoptar un **comportamiento ético**: **no** robar datos de clientes que pudiera quedar expuestos, **no** hacer compras en nombre de otra persona, etc. Es clave cuidar la *credibilidad* de estas pruebas. Es también recomendable que las haga *alguien que no intervino en la construcción* del sistema (análogo al rol de QA: el desarrollador prueba, hace regresiones y pruebas unitarias; luego otro intenta romperlo).

¿Quién hace los pentests y por qué importan?

El profesor remarca que esta metodología es **base de lo que hoy se usa para testear**, fundamental si se aspira a trabajar en equipos de *Red Team*, *AppSec* o *InfraSec*. Para cualquier sistema mediano o grande hoy se **exige** algún tipo de pentesting, sobre todo en los

componentes *core*, y que no tengan vulnerabilidades críticas o altas (criticidad definida por los **CVE**). Por eso es importante la organización **MITRE**. A veces se contrata un proveedor; si la empresa es suficientemente madura y grande, se hace internamente con gente experta.

6 Metodología (informal)

Es la metodología vista “por encima” la clase anterior. A grandes rasgos, sus pasos son:

1. **Determinar y cuantificar objetivos.** Preparación: qué vamos a hacer, qué alcance tiene, con qué tiempo contamos, cuál es el propósito. *Ejemplo: obtener información de clientes.*
2. **Encontrar cierta cantidad de vulnerabilidades o buscarlas durante un período de tiempo.** El estudio se conduce **desde el punto de vista de un atacante**. Aquí se arma el *plan de ataque*: qué herramientas se usan, dónde se ubican, qué equipo está a cargo, quiénes participan y quiénes están al tanto.
3. **Estudiar y categorizar los hallazgos.** *El éxito de una prueba de penetración proviene del análisis de los hallazgos*: permite detectar problemas recurrentes y corregir sistemáticamente **familias** de problemas.

Aclaraciones de la clase

- **Lower line / aviso:** muchas veces *no* se avisa a los equipos de desarrollo u operativos que va a ejecutarse un pentest (para que la prueba sea realista).
- **Condición de éxito:** el objetivo tiene una condición que determina si la prueba tuvo éxito o no. Se ataca hasta lograr el objetivo o hasta que **se vence el tiempo** definido; ahí termina la prueba.
- **Es cíclico:** como casi todos los procesos de seguridad, los hallazgos se reincorporan al ciclo y se encadenan con otras pruebas. Una vulnerabilidad suele tener un *patrón de comportamiento*: aparece en distintos componentes (porque se desarrollaron igual o usan la misma dependencia vulnerable).

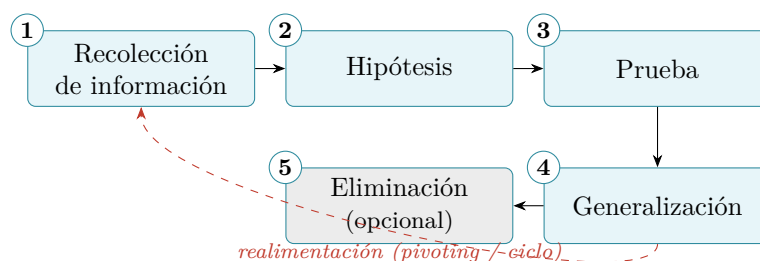
7 Metodología de Hipótesis de Falla

¡Importante para el parcial!

“Esta metodología es importante, es súper importante que la entiendan y la sepan: es la base de lo que hoy se usa para testear.”

— Prof. Leandro Suárez, en clase Es la metodología en la que se basa el pentest. Si bien en la práctica no se respeta al detalle, es la **estructura** de cómo debería funcionar un pentest a grandes rasgos.

Consta de **cinco etapas**, la última opcional:



1. **Recolección de información** — para conocer el sistema y su funcionamiento.
2. **Hipótesis** — asumir la existencia de vulnerabilidades concretas.
3. **Prueba** — probar las vulnerabilidades.
4. **Generalización** — generalizar patrones y hallar otras vulnerabilidades del mismo tipo.
5. **Eliminación (opcional)** — determinar los pasos necesarios para eliminarlas.

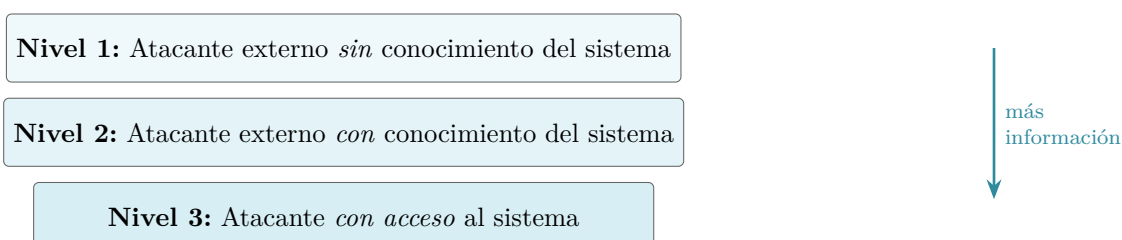
7.1 Etapa 1 — Recolección de información

El objetivo es **componer un modelo del sistema y sus partes**, interiorizarse con él para saber “dónde poner el ojo”.

- **Buscar discrepancias.** Cuando el sistema evoluciona y la documentación no refleja esa evolución, aparece una posible *puerta de entrada*: la documentación dice que se comporta de una forma pero ya no lo hace.
- **Revisar interfaces:** conexiones con sistemas externos, proveedores, otros sistemas, bases de datos.
- **Conocer bien el sistema** usando documentación de diseño y manuales (si están disponibles). A mayor documentación, mejor *insight*. Especialmente, buscar secciones poco especificadas o ambiguas: ahí pudo tomarse alguna decisión cuestionable $\Rightarrow$ posible puerta de entrada.
- **Ver manejo de privilegios y tipos de cuentas:** enumerar usuarios; buscar cuentas con *superadmin*/supervisor (las de mayor poder, generalmente sin límites), y **cuentas huérfanas** o con exceso de permisos.
- **El ambiente:** conseguir nombres de usuarios y servidores, en qué servidores corre, la **estructura de red** del sistema y su configuración.
- **Vulnerabilidades conocidas genéricas** que puedan ser compatibles con la tecnología que usa el sistema.

7.2 Punto de partida

Identifica la **información inicial con la que cuenta el equipo** (el supuesto atacante). Se distinguen **niveles incrementales**:



Dependiendo del tipo de prueba, algunos niveles son **irrelevantes**:

- Durante el **diseño**, se ignora el *primer* nivel.
- En sistemas con **registro abierto**, se ignora el *segundo* nivel.

7.3 Etapa 2 — Hipótesis

Aquí se usa toda la información recabada para **buscar posibles vulnerabilidades**, tratando de encontrar combinaciones que expongan una vulnerabilidad.

Examinando políticas y procedimientos

- Buscar **inconsistencias** (incluidas las inconsistencias entre *políticas* y *mecanismos* que las implementan).
- Los **procedimientos pueden no cumplirse** en la práctica.

Examinando implementaciones

- Utilizar **modelos de vulnerabilidades**.
- Revisar vulnerabilidades comunes/conocidas (**vulnerability scanning**, p. ej. portmapper).
- Usar manuales para **exceder límites**, **omitir pasos** de las secuencias, etc.

Vectores de hipótesis específicos (slide de Hipótesis)

- **Seguridad en APIs:** política de desarrollo de *microservicios* y *Software Supply Chain (API Management)*. Cuantos más servicios/APIs, más “cajitas” atacables; si cada una está a cargo de un equipo distinto, puede no haber una política de seguridad homogénea.
- **RASP (Runtime Application Self Protection):** pueden estar *mal implementados*. Son herramientas que controlan los binarios del sistema para que no cambien durante la ejecución; si un malware intenta modificar o reemplazar un binario, lo detectan y pueden alertar o tirar abajo el sistema.
- **Verificar Federadores:** vulnerabilidades conocidas sobre protocolos. Se implementan protocolos como *OAuth* (p. ej. login con Google); al ser protocolos abiertos y antiguos, vienen siendo atacados desde su origen y una mala implementación los vuelve vulnerables.
- **Software EOL (End Of Life):** software fuera de soporte, sin actualizaciones de seguridad o desactualizado. Es “muy jugoso” a la hora de pentestear.

Examinando mecanismos y comparando con otros sistemas

- Los **mecanismos** pueden estar mal implementados, el ambiente donde corren puede introducir errores, o pueden directamente no ser seguros.
- **Comparar con otros sistemas:** *sistemas parecidos tienen problemas parecidos*.

Resultado de la etapa de hipótesis

Una lista de posibles vulnerabilidades que asumimos que tiene el sistema.

Anécdota: Supply Chain y NPM

El **Software Supply Chain** es hoy *top 3* del OWASP Top Ten para aplicaciones web (“cuando yo empecé a trabajar ni se hablaba”). Abarca **dependencias vulnerables**, infiltraciones en flujos de *CI/CD* (plugins/dependencias que ejecutan chequeos) y *dependency confusion*: una dependencia que parece genuina pero es otra, terminando en la descarga de malware. En **NPM** el problema se agrava por las *dependencias transitivas*: NPM baja *todo el árbol* de dependencias, así que si una dependencia muy usada queda afectada, tu aplicación es vulnerable “de por sí”. Han ido apareciendo otros gestores que evitan bajar lo innecesario.

7.4 Etapa 3 — Prueba

Priorizar

Priorizar la lista de posibles vulnerabilidades. *Depende del objetivo de la prueba*; por lo general se prioriza por **nivel de acceso requerido**.

Cómo probar la existencia de la vulnerabilidad

- **La mejor manera:** como resultado de analizar la *documentación* o el *comportamiento*.
- **Otra manera: intentar explotarla.** Es el *último recurso* y el *menos eficiente*, porque hay que lograr un exploit como lo haría un atacante. **Excepción: *Pivoting*** (explicado más adelante).

Demostrar = explotar (con cuidado)

Hoy se asume que, para *demostrar* una vulnerabilidad, hay que tratar de **explotarla** (es lo que da peso al hallazgo: “probé la existencia explotándola”). Es el origen de las *PoC* (*Proof of Concept*) en pentesting. Pero hay que tener mucho cuidado, sobre todo en sistemas productivos.

Diseñar el test — lo menos intrusivo posible

Algunas vulnerabilidades pueden **denegar el servicio** (DoS) y atentar contra la tríada **CIA** (Confidentiality, Integrity, Availability). *Proceso normal*:

1. **Resguardar los datos** de todo el sistema (backup).
2. **Documentar** los requerimientos para detectar la vulnerabilidad.
3. **Intentar detectar** la vulnerabilidad.

La prueba DEBE ser REPETIBLE y CONCLUSIVA

Si funciona una sola vez y no se puede replicar (algo esporádico), **no concluye** que haya vulnerabilidad. *Repetible* significa que si el pentester puede hacerlo, naturalmente un atacante también podrá.

Anécdotas sobre el peligro de explotar:

- **Chernóbil:** era una *prueba controlada* (ver cómo respondía el núcleo en un escenario sin energía) y terminó en desastre. “Controlado” no garantiza seguridad.
- **Nmap:** programa de Linux (aparece en *Matrix*: Trinity lo corre) que escanea interfaces y puertos y arma la **topología de la red** (un *discovery* de la red). El profesor vio a gente **tirar abajo la red de la oficina** corriendo un Nmap, afectando la disponibilidad.

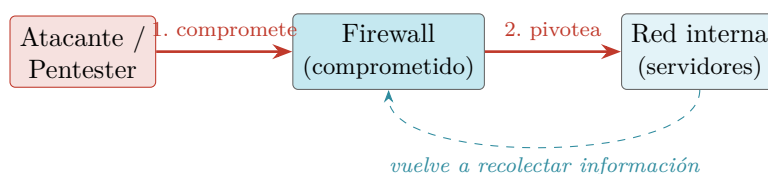
7.5 Pivoting

Pivoting

La existencia de una vulnerabilidad probablemente **proporcione más información** (ejemplo: acceso al Sistema Operativo). En ese caso **se debe volver a la fase de recolección de información**. *Pivoting* es el **uso de un punto de acceso comprometido para continuar un ataque**.

Ejemplo: se compromete el **firewall** y desde allí se continúa atacando la **red interna**.

Esta es la **excepción** a “explotar es el último recurso”: encadenar una vulnerabilidad con otra hace ganar cada vez más posibilidades y ofrecer más “daño”. El diagrama recrea la idea del firewall comprometido como trampolín:



7.6 Etapa 4 — Generalización

A medida que las pruebas resultan exitosas, **emergen patrones**: “esto se está dando muy seguido, tiene que haber algo por acá”. La idea es *conectar puntos*.

Dos vulnerabilidades combinadas = problema grave

A veces dos vulnerabilidades combinadas constituyen un problema grave. *Ejemplo de la slide:*

- Una **cuenta de invitado** habilitada permite **conexión remota**.
- Un **buffer overflow local** permite **derechos de administrador**.

⇒ A través de una cuenta de invitado se pueden lograr **permisos de administrador**. Una vulnerabilidad puede no ser crítica por sí sola, pero *combinada* con otra causar un daño importante.

La “mano” del pentester

“Constituye la parte más importante de la prueba.”

— Prof. Leandro Suárez, en clase Es donde el pentester identifica cosas, las combina y llega a conclusiones más avanzadas. Su función es encontrar la mayor cantidad de vulnerabilidades y “fingir” el mayor daño (entre comillas, no es un daño real) para darle ese *insight* a la empresa y que pueda corregir toda esa superficie de ataque.

7.7 Etapa 5 — Eliminación (opcional)

Por lo general **sólo se incluyen recomendaciones** (no es obligatorio, pero se suele hacer): quien ejecuta la prueba *no suele ser* el diseñador/desarrollador, así que el dueño del sistema termina decidiendo qué hacer.

Es importante que queden claros el **contexto**, **los detalles** y el **mecanismo de explotación**:

- Para poder **corregir** el sistema.
- Para poder **impedir** / **monitorear** mientras tanto.

- Para **verificar si fue explotado** y, además, poder *reproducir* la prueba luego del fix y comprobar que ya no es vulnerable.

El reporte de pentesting

Todo termina en un **reporte** donde se detallan las vulnerabilidades encontradas y cómo se propone eliminarlas (como *recomendaciones*). A veces la remediación es tan simple como actualizar la versión de una dependencia; otras veces es más abstracta (“manejar bien la autorización”). Hay gente dedicada a documentar la remediación de los **CVE**.

8 Ejemplo 1 — Michigan Terminal System (MTS)

Esta prueba de penetración contaba con la **aprobación de los administradores**, que fueron partícipes de la prueba.

El **Michigan Terminal System (MTS)** es un sistema operativo que corre en **mainframes IBM 360/370** (computadoras enormes, en general con aplicaciones bancarias).

- **Objetivo de la prueba:** obtener acceso a las **estructuras de control** que no debería permitirse.
- **Nivel inicial:** cuenta autorizada (**nivel 3**, atacante con acceso al sistema).

8.1 Paso 1 — Recolección de información

Se aprenden los detalles del **mecanismo de control**: al correr un programa, la memoria se divide en **segmentos**.

Segmentos 0–4: supervisor, programas de sistema y estado (protegidos por hardware)

Segmento 5: área de trabajo del sistema, info. del proceso (incl. nivel de privilegio)

el proceso NO puede alterar esta información

Segmentos 6+: información del proceso (el proceso modifica libremente)

El segmento 5 (el codiciado)

Está protegido por el **sistema de protección de memoria virtual**:

- **Modo sistema (kernel):** el proceso puede acceder, modificar datos y ejecutar funciones privilegiadas.
- **Modo usuario:** el segmento 5 *no se mapea* al espacio de direcciones del usuario.

Un proceso corre en **modo usuario**; desde ahí realiza **system calls**, que corren en **modo sistema** (de forma controlada).

Ejecución de un system call

1. El código del system call **revisa los parámetros**.
2. Especialmente, revisa que los **punteros** pertenezcan a zonas *accesibles por el proceso*.
3. Los parámetros se construyen como una **lista de direcciones en el segmento de usuario**.
4. La lista de parámetros se pasa como una **dirección en un registro**.

8.2 Paso 2 — Hipótesis

La hipótesis

¿Qué ocurriría si la dirección de un parámetro apunta a la lista misma de parámetros? (un puntero que apunta a sí mismo).

Sin controles extra, sería posible que algún system call **escriba la dirección del parámetro después de haber pasado la validación**. Si se puede controlar qué valor escribir, técnicamente sería posible **leer o escribir cualquier zona del segmento 5** (el segmento que no se debería poder tocar). Es un *TOCTOU*: se valida y luego se altera.

8.3 Paso 3 — Prueba

Buscar un system call que: **(a)** use la convención de llamada estudiada, **(b)** tenga al menos dos parámetros y altere uno, y **(c)** permita que ese parámetro se altere con cualquier valor.

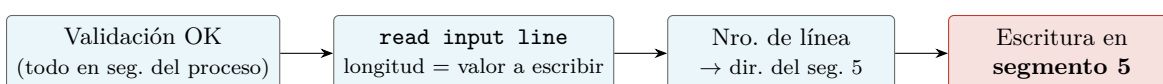
El candidato elegido: line input

`line input` lee una línea de texto y **retorna dos valores**: el *número de línea* y la *longitud de línea*.

Setup del exploit: hacer que la *dirección* donde se almacenará el **número de línea** sea la de la **longitud de línea**.

Ejecución del exploit

1. El system call **valida los parámetros**: todos pertenecen al segmento del proceso $\Rightarrow$ pasa la validación.
2. Ejecuta el método `read input line`: la **longitud** de la línea ingresada es el *valor a escribir*.
3. El **número de línea** se guarda según la lista de parámetros; se logra que sea una **dirección del segmento 5**. Esto **reescribe la dirección del otro parámetro**.
4. La **longitud** se guarda según la lista de parámetros $\Rightarrow$ esto **escribe el valor en el segmento 5**.



8.4 Paso 4 — Generalización

- **No** se puede escribir en los segmentos 0–4 (protegidos por hardware).
- **Pero** en el segmento 5, el nivel de privilegio determina si pueden hacerse **llamadas de nivel supervisor** (en lugar de system calls).
- Y una función de nivel supervisor **apaga la protección por hardware**.

Conclusión del MTS

Esta vulnerabilidad permite que un atacante **modifique cualquier dirección de cualquier segmento, controlando completamente la computadora**. (El profesor aclara que era otra época, sin los conceptos fuertes de seguridad de hoy; este ejemplo proviene del material de Pablo.)

9 Ejemplo 2 — Ataque externo (ingeniería social)

- **Objetivo:** determinar si las medidas de seguridad de una empresa eran efectivas para evitar que un atacante ingrese a un sistema.
- **Enfoque:** el test se enfoca en **políticas y procedimientos**, tanto técnicos como **no técnicos**.

Premisa: la ingeniería social

Este pentest está fuertemente apalancado en **ingeniería social**, que asume que es **mucho más fácil atacar y vulnerar a las personas que a los sistemas**. Se aprovecha la *urgencia* y la *confianza*: “estoy apurado, lo necesito para la auditoría” hace que la gente entregue información “en base a urgencia”.

9.1 Paso 1 — Recolección de información

- **Búsqueda en internet:** nombres de empleados y directores; teléfono de la sucursal local. De ahí se consiguió el **reporte anual público** (la empresa cotizaba en bolsa).
- **Reconstrucción de estructura:** partes del **organigrama**, nombres de proyectos y personas que trabajan en cada uno.

Obtención del listado de empleados (encadenamiento de pretextos)

1. El tester se hizo pasar por un **nuevo empleado** y aprendió que para enviar cualquier documento se necesita: **número de empleado y centro de costos**.
2. Llamó a la **secretaria del director** (sobre quien más información había recabado):
 - Primero, haciéndose pasar por un empleado, consiguió el **número de empleado** del director.
 - Luego, haciéndose pasar por un **auditor**, obtuvo el **centro de costos**.
3. Con esos datos, **solicitó enviar el listado** de empleados a una consultora externa.

9.2 Paso 2 — Hipótesis

Hipótesis

Los **empleados nuevos no conocen todos los procesos y controles** de la compañía. Por lo tanto, es posible que un nuevo empleado **brinde información sensible**.

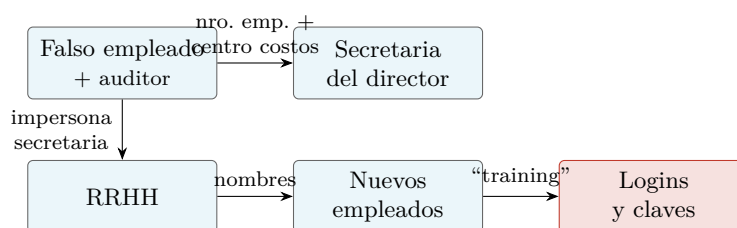
9.3 Paso 3 — Prueba

1. El tester llamó a **RRHH** impersonando a la **secretaria de un director**: se quejó de que no le habían informado los nuevos ingresos y los pidió. Obtuvo los **nombres de los nuevos empleados**.
2. El tester llamó a los **nuevos empleados** haciéndose pasar por el **operador del centro de cómputos** y les brindó un “entrenamiento de seguridad por teléfono”. Durante el entrenamiento obtuvo: **tipos de sistemas usados, número de usuarios, logins y claves**.

El truco para sacar la clave

El nuevo empleado confía: alguien con su número lo llamó y trabaja para la empresa. En medio del “entrenamiento”, el tester critica la contraseña (“es muy débil, no usaste símbolos

ni mayúsculas, no cumple la política”), se ofrece a cambiarla y pide: “decime cuál es y yo la cambio”. Así obtiene la clave. *Resultado:* se probó que los empleados nuevos no estaban al tanto de los procesos/controles y brindaban información sensible. La hipótesis se cumplió.



10 Para discutir: ¿cuán válidos y de qué calidad son los pentests?

10.1 ¿Cuán válidos son los tests de penetración?

- **No sustituyen** una buena especificación, diseño, implementación y pruebas (unitarias, de integración, cobertura, principios de diseño de seguridad).
- Son una técnica **importante para probar un sistema luego de ser instalado**. *Idealmente no sería necesario; en la práctica, sí.*
- Encuentran **problemas introducidos por la interacción** del sistema con los usuarios y el ambiente, que suelen quedar *fuera* del análisis y las pruebas normales. *“Siempre algo aparece.”*

10.2 ¿Qué determina la calidad de una prueba de penetración?

- La metodología de hipótesis de fallas **depende de la capacidad de los testers** para formular hipótesis (su “visión”).
- **No es sistemática:** no provee una forma sistemática de revisar un sistema (a diferencia de la verificación formal).
- Los resultados de un test sirven sólo **marginalmente** para otros: *cada test es un mundo aparte*, caracterizado por el sistema, la gente y muchos factores. Hay herramientas que automatizan *algunos* aspectos, pero no todos.

Un pentest sirve para ese sistema y ese momento

Un pentest sirve **para ese sistema en ese momento**: un mes después la API, el comportamiento o las versiones cambiaron y la gran mayoría de las pruebas pudo quedar *obsoleta*. Por eso no se puede generalizar como sí se hace con la verificación formal.

11 Síntesis

Concepto	Idea clave
Verificación formal	Prueba matemática; prueba <i>ausencia</i> de vulnerabilidades (con todos los factores); costosa e impráctica.
Pentesting	Prueba <i>existencia</i> , nunca ausencia; económico; ethical hacking; ataca el sistema como un todo.
Hipótesis de Falla	5 etapas: Recolección → Hipótesis → Prueba → Generalización → Eliminación (opcional).
Recolección	Modelar el sistema, discrepancias, privilegios/cuentas, ambiente y red.
Hipótesis	Inconsistencias políticas/mecanismos; APIs/Supply Chain, RASP, federadores (OAuth), EOL.
Prueba	Priorizar por acceso; demostrar explotando (último recurso); test <i>repetible y conclusivo</i> ; backup + documentar.
Pivoting	Usar un acceso comprometido para seguir atacando; volver a recolectar.
Generalización	Conectar patrones; combinar vulnerabilidades (lo más importante).
Eliminación	Recomendaciones; contexto/detalle/mecanismo para corregir, monitorear y verificar.

Lectura recomendada

Capítulo 23 (secciones 1–2) — *Computer Security: Art and Science*, Matt Bishop.

OSSTMM — *Open Source Security Testing Methodology Manual*,
<http://www.isecom.org/osstmm/>